

Mobile Development

How does Bart handle mobile app development?

Nowadays there are dozens of ways to generate native builds for Android and IOS platforms, so I focus on the internal workings and functionality of the application, using the tech stack and tools that I am the most familiar with. My current preferred tech stack is React with Typescript and Sass for the UI, and Node.js with Typescript or Python with FastAPI for the backend. With that said I do also design applications with a “mobile-first” approach, which means that I design an application to work on a small screen (mobile device) – first, and make sure the core functionality works in that limited framework, and then build and add additional options to tablet and desktop layouts. Within the context of web-based applications, this means designing HTML and CSS systems that are responsive and scale fluidly. For larger applications, it often means generating separate builds for mobile executables that can run natively on different devices. I’ve personally built a few personal projects with Swift for iOS and a combination of Flutter & Dart for cross-platform mobile native applications, and web applications that use the same code base. I’m also currently exploring other tech stacks such as Tauri, Rust, Go and HTMX.

How do you think about optimizing mobile experiences for activation, engagement, or retention? *

Thoughts on mobile design and development:

I always design and develop with a 'mobile first' approach. This means focusing on having responsive layouts, server-based heavy lifting, and adding complexity for desktop - rather than pruning complexity for mobile.

However, when it comes to mobile experiences directly:

Activation:

First impressions and onboarding are absolutely critical. Activation starts before a user even taps a button — it's about the visual clarity of the interface signaling "this is worth your time." This means branding. Are the icons in your app store custom designed to convey a consistent brand identity across all device sizes, not just relying on re-scaled defaults or lo-res compressions. Does the app appear visually consistent between Android and iOS? Users begin to build their trust and comfort level with these first few impressions and interactions. You want to have thoughtful considered implementations of every aspect. Users also regularly switch between one device, one platform and another. Inconsistencies, or missing functionality stands out, and lowers users trust and activation. This is actually not a huge undertaking – but it does take focus and diligence to reach a base level of professionalism that users have come to expect. Also – if your on-boarding process is too cumbersome, doesn't use modern standards, or is not responsive or not thoroughly tested, you lose users – who may never come back. I've seen too many organizations treat both of these issues as low-level importance, when actually they are completely critical.

Engagement:

If the app is providing the information or service that users are looking for, then increasing engagement comes down to reducing friction.

First – reduce cognitive friction. For example – users should be able to navigate the site – without having to THINK about how to navigate the site. If an icon is not immediately recognizable for at least 90% of users – then you need a different icon. If users can't get to what they want within a click or two – you will lose them. Some users have better eyesight than others. Not everyone is 26 anymore. Some users are on their phone in a dark room early in the morning before they've put their glasses on– others are out on the boardwalk on a bright sunny day. Also – this extends to text size, contrast ratios and light/dark color schemes. Let users decide how they want to experience your content.

Second – reduce time friction. This requires both technical solutions, and even more critical - design solutions. API latency can be reduced by using clever API design, contracts, and techniques. (only the minimum data needed should be sent). API calls can be reduced by using clever UI state management. (UIs shouldn't need to make multiple calls to an API just because the user navigated to another page). But equally important are the graphical cues sent to the user. Let users know they are waiting for data to be retrieved. Be responsive. If a user clicks a button and it doesn't visually respond, they assume the app is broken or dead. All buttons should have 'onClicked' visual state to let users know the app received their push of the button. And spinner graphics should be deterministic – a progress bar demonstrates respect for users time and intelligence. A spinning beach ball has about a second or two before users will leave. With that said, spinners are very useful when there are small lags – if a button needs to call an API that can't be called ahead of time, adding a spinner on that button directly can actually feel very responsive to users – because it lets them know things are happening.

Retention:

If you've successfully provided a trust-worthy, branded experience that is consistent across media and platforms, and you've reduced cognitive and time friction, then retention comes down to personalization. Users expect and will return to an app that remembers them. It should remember their preferences. Users should be able to easily save their preferences and / or filters and reuse them later on. It should have a 'single-source-of-truth' and preferences that persistence across devices. If a user prefers dark mode on their phone – they probably also prefer dark mode on their desktop. Paying attention to the nuances of customer experience can make all the difference when it comes to customer loyalty and retention.